

A survey of race bug detection techniques for multithreaded programmes

Shin Hong and Moonzoo Kim^{*,†}

Computer Science Department, KAIST, South Korea

SUMMARY

As multithreaded programmes become popular to fully utilize multicore CPUs, many race bug detection techniques have been developed to find concurrency errors in multithreaded programmes effectively. Because these techniques have different views on target programme execution and detect race bugs of various types, it is difficult to characterize, compare and improve race bug detection techniques. This paper presents a formal *execution model*, which can uniformly represent various views of race bug detection techniques on target programme execution. Then, this paper classifies 43 race bug detection techniques according to their target race bugs. We classify race bugs on whether or not a bug violates *operation block* specification and/or *data association* specification. This survey provides researchers with a clear top-down view of various race bug detection techniques. In addition, the concrete examples of various race bugs in this survey can help field engineers avoid race bugs in their multithreaded programmes. Copyright © 2014 John Wiley & Sons, Ltd.

Received 23 April 2013; Revised 7 November 2014; Accepted 10 November 2014

KEY WORDS: race bug detection; race condition; concurrency bug; survey

1. INTRODUCTION

Developing bug-free multithreaded applications is challenging because of non-deterministic thread scheduling, which generates a large number of diverse programme behaviours to be checked for correctness. To make multiple threads behave correctly, multithreaded applications enforce synchronization amongst the threads. A mistake in synchronization, however, can cause *race bugs*, which make a target programme execute abnormally with unintended accesses to shared resources. To address this problem, many race bug detection techniques have been developed with predefined *race bug patterns*. These techniques check if execution trace or programme code matches the patterns through static or dynamic analyses to identify race bugs.

Despite the wealth of literature on race bug detection techniques, it is difficult to analyse and compare them due to their use of various race bug patterns. This is because different race bug detection techniques define their race bug patterns using different methods, such as the programme analysis technique (e.g. dynamic analysis, static analysis, etc.), views on the programme execution (e.g. totally ordered execution, partially ordered execution, etc.), target synchronization primitives (e.g. binary lock, wait/notify, etc.) and target memory models (weak memory model, sequentially consistent memory model, etc.). Furthermore, these techniques define the target race bug patterns using different terminologies, which make the description of the race bug detection techniques difficult to understand.

In this paper, as the first step to alleviate the aforementioned problems, we define a formal *execution model*, which provides an abstract view on the programme execution (Section 3). By using the proposed *single* execution model, we can define various target race bug patterns from different

^{*}Correspondence to: Moonzoo Kim, Computer Science Department, KAIST, South Korea.

[†]E-mail: moonzoo@cs.kaist.ac.kr

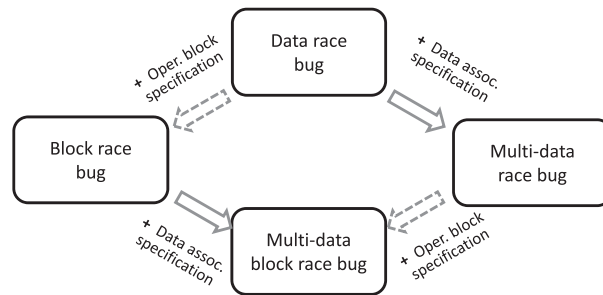


Figure 1. Relationship amongst the four classes of race bug detection techniques.

Table I. Data race bug detection techniques.

Technique	<i>thread()</i> (DR1)	<i>operand()</i> (DR2)	<i>conflict()</i> (DR3)	$\rightarrow_S$ (DR4)
Acculock [30]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock
Choi <i>et al.</i> [31]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, thread create/join
Chord [42]	Static approximation	Code analysis	Read–read, read–write, write–read, write–write	Lock
Eraser [1]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock
FastTrack [26]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, thread create/join
GUARD [32]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, barrier
LiteRace [34]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, atomic oper. message send/receive
O’Callahan <i>et al.</i> [35]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, message send/receive
Racer [28]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, thread create/join
RacerX [29]	Static approximation	Code analysis	Read–write, write–read, write–write	Lock
RaceTrack [36]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, thread create/join
RACEZ [37]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock
RccJava [43]	Static approximation	Code analysis	Read–read, read–write, write–read, write–write	Lock
RELAY [44]	Static approximation	Code analysis	Read–write, write–read, write–write	Lock
SOS [38]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, thread create/join
ThreadSanitizer [39]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, message send/receive
TRaDe [27]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock

race bug detection techniques. Then, we compare and classify the techniques according to their target race bugs. We classify race bugs depending on whether or not a bug violates *operation block* specification and also depending on whether or not the bug violates *data association* specification (Section 2). Figure 1 shows the relationships amongst the following four classes of race bugs:

- *Data race bug*. Operations that manipulate a shared variable in different threads without synchronization (Section 4)
- *Block race bugs*. Operations that violate an operation block specification (Section 5)
- *Multi-data race bugs*. Operations that violate a data association specification (Section 6)
- *Multi-data block race bugs*. Operations that violate both operation block specifications and data association specifications (Section 7)

We classify race bug detection techniques that do not utilize operation block nor data association specification as *data race bug* detection techniques (Section 4). Techniques that use the lockset algorithm [1] belong to this class. We classify race bug detection techniques that utilize operation block specification but not data association specification as *block race bug* detection techniques (Section 5). This class includes atomicity violation detectors and serializability violation detectors. On the other hand, *multi-data race bug* detection techniques utilize data association specification but not operation block specification (Section 6). *Multi-data block race bug* detection techniques utilize both operation block and data association specifications (Section 7).

We have classified 43 race bug detection techniques in terms of the aforementioned four classes: data race bug detection techniques, block race bug detection techniques, multi-data race bug detection techniques and multi-data block race bug detection techniques. In addition, we have compared the key mechanisms used by the race bug detection techniques to identify the target race bug patterns in programme execution or code (Tables I–IV). Thus, this survey can help researchers obtain a clear top-down view of various race bug detection techniques for multithreaded programmes.

For example, suppose that we would like to know if Velodrome [2] can detect the race bugs that AVIO [3] cannot detect. The answer to the question is not simple because these techniques define race bugs using different notions of correctness, namely, conflict serializability in Velodrome

Table II. Block race bug detection techniques.

Technique	<i>thread()</i> (BR1, BR4)	<i>operand()</i> (BR3, BR5)	<i>conflict()</i> (BR6)	$\rightarrow_S$ (BR7)
Atom-Aid [9]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write'	Lock
AtomRace [10]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write'	Lock
AVIO [3]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write'	Timestamp
Block-based algorithm [14]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write'	Lock, message send/receive
CTrigger [24]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write'	Barrier, lock, thread create/join
DefUse [40]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Timestamp
FALCON [41]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Timestamp
Kivati [11]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write'	Timestamp
McPatom [5]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock, wait/notify
PENELOPE [4]	Runtime thread identifier	Runtime memory address	Read–write, write–read, write–write	Lock

Table III. Multi-data race bug detection techniques.

Technique	$thread()$ (MR1)	S_{DA} (MR2)				$conflict()$ (MR3)	$\rightarrow S$ (MR4)
		Ref	Sym	Tran	Type		
MultiRace [20]	Runtime thread identifier	○	○	○	Pairs of mem. locations	Read–write, write–read, write–write	Lock, thread create/join
MUVI-Eraser [22]	Runtime thread identifier	○	×	×	Ordered pairs of mem. locations w/ access types	Read–write, write–read, write–write	Lock
Object data race detection [21]	Runtime thread identifier	○	○	○	Pairs of mem. locations	Read–write, write–read, write–write	Lock

Table IV. Multi-data block race detection techniques.

Technique	$thread()$ (MBR1, MBR4)	S_{DA} (MBR3, MBR5)				$operand()$ (MBR3, MBR5)	$conflict()$ (MBR6)	$\rightarrow S$ (MBR7)
		Ref	Sym	Tran	Type			
Atomic-set serial. violation detector [6]	Runtime thread identifier	○	○	○	Pairs of mem. locations	Runtime mem.	Read–write, write–read, write–write	Timestamp
AtomTracker [33]	Runtime thread identifier	○	○	×	Pairs of mem. locations	Runtime mem	Read–write, write–read, write–write	Timestamp
Block-local atomicity [16]	Static approximation	○	×	×	Ordered pairs of mem. locations	Code analysis	Read–read, read–write, write–read, write–write	Lock
ColorSafe [13]	Runtime thread identifier	○	○	○	Pairs of mem. locations	Runtime mem.	Read–write, write–read, write–write	Timestamp
Commit node [17]	Runtime thread identifier	○	○	×	Pairs of mem. locations	Runtime mem.	Read–write, write–read, write–write	Lock, message send/receive
COPPER [8]	Runtime thread identifier	○	○	×	Pairs of mem. locations	Runtime mem.	Read–write, write–read, write–write	Lock, wait/notify, thread create/join
Dias <i>et al.</i> [7]	Static approximation	○	×	×	Ordered pairs of mem. locations w/ access types	Code analysis	Read–read, read–write, write–read, write–write	Lock
HAVE [15]	Runtime thread identifier	○	○	×	Pairs of mem. locations	Runtime mem., code analysis	Read–write, write–read, write–write	Lock, message send/receive
High-level data race detection [18]	Runtime thread identifier	○	○	×	Pairs of mem. locations	Code analysis	Read–read, read–write, write–read, write–write	Lock
Marathon [12]	Runtime thread identifier	○	○	○	Pairs of mem. locations	Runtime mem.	Read–write, write–read, write–write	Timestamp
Method-consistency [19]	Static approximation	○	○	×	Pairs of mem. locations w/ access type	Code analysis	Read–read, read–write, write–read, write–write	Lock
MUVI-AVIO [22]	Runtime thread identifier	○	×	×	Ordered pairs of mem. locations w/ access type	Runtime mem.	Read–write, write–read, write–write	Timestamp
Velodrome [2]	Runtime thread identifier	○	○	×	Pairs of mem. locations	Runtime mem.	Read–write, write–read, write–write	Lock

and access interleaving (AI) invariants in AVIO. In addition, Velodrome is designed for Java programmes with `synchronized` blocks as target synchronization primitives, whereas AVIO analyses C programmes with `lock()`/`unlock()` as target synchronization primitives. Thus, it is difficult to compare the race bug detection capabilities of these two techniques. In contrast, our classification shows that AVIO is a *block race bug* detection technique, whereas Velodrome is a *multi-data block race bug* detection technique. This information allows us to simply answer ‘yes’ to the aforementioned question because the block race bug class is a strict subset of the multi-data block race bug class (Section 8.3). As another example, from Table II, we know that PENELOPE [4] aims to detect the same class of race bugs targeted by AVIO. However, PENELOPE detects more race conditions as race bugs than AVIO because PENELOPE considers to write operations of

different threads as a conflict, whereas AVIO does not (see the fourth column of Table II). In addition, PENELOPE takes the effect of lock synchronization into account, whereas AVIO does not (see the last column of Table II). Thus, PENELOPE and AVIO might detect different block race bugs.

The contributions of this paper are as follows:

- The classification of the 43 techniques provides a systematic overview on race bug detection techniques, which help researchers understand and improve race bug detection techniques.
- The systematic characterization and concrete examples of a broad range of race bugs can help field engineers avoid race bugs in their multithreaded programmes.

This paper is organized as follows. Section 2 gives an overview of our classification scheme. Section 3 presents a formal execution model based on which classes of race bugs are defined. Sections 4–7 describe each of the four classes of race bugs and corresponding race bug detection techniques. Section 8 illustrates the relation between the race bug classes. Section 9 briefly reviews other survey papers on concurrency bug detection techniques. Finally, Section 10 concludes this paper with future work.

2. OVERVIEW OF RACE BUG CLASSIFICATION

A *race bug* refers to a multithreaded programme fault that causes race conditions, which in turn make the programme execute incorrectly (i.e. reaching an invalid state). A multithreaded programme has a *race condition* if an execution order of two operations is not fixed (i.e. non-deterministic) and their execution result varies depending on the order.[‡] Race bug detection techniques detect race bugs by checking if the target programme can violate synchronization requirements, which must be satisfied to control multiple threads correctly. Race bug detection techniques typically work in the following three steps:

1. Modelling concurrent programme behaviours and synchronization requirements;
2. Identifying operations that are involved with *race conditions*; and
3. Checking if the identified operations can violate the synchronization requirements.

In this paper, we classify race bug detection techniques according to the usage of *operation block* specifications and *data association* specifications, which are orthogonal synchronization requirements for correct multithreaded executions.

- Operation block (S_{OB} of Definition 1 in Section 3.2)
An operation block specification is defined as a sequence of consecutive operations of a thread, which are intended to execute without interference from other threads. A programme is considered to *satisfy* an operation block specification if a specified operation block always results in a state (regardless of interference from other threads) that is equivalent to the resulting state of the non-interfered operation block.
Race bug detection techniques receive an atomic region specification that specifies an operation block in the target programme.[§] Some race bug detection techniques provide annotation mechanisms for users to specify atomic regions on the target programme code [2, 4–8]. Other techniques automatically infer an operation block specification by using predefined code patterns or execution patterns [3, 9–15]. Still, other techniques simply consider executions of function/method body of certain types as operation blocks [16–19].
An operation block corresponds to an execution of ‘atomic code block’ [2], ‘unit of work’ [6] or ‘transaction’ [17] that occurs in the literature. Instead of using these terms, we decided

[‡]Race conditions do *not* necessarily cause incorrect programme executions. Programmers may utilize race conditions for better processor utilization and responsiveness to requests, because race conditions enable various thread execution orders.

[§]Atomic region specification should not be confused with atomic constructs such as `synchronized` blocks in Java.

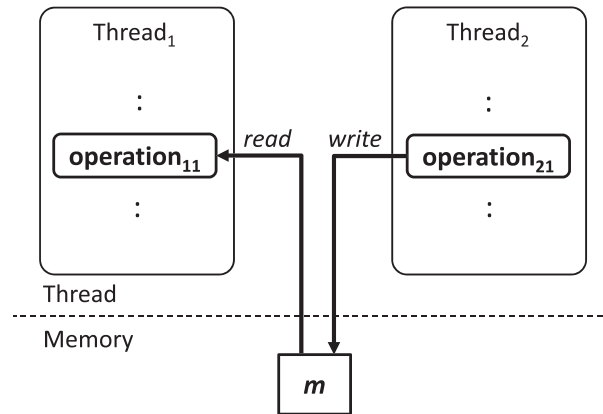


Figure 2. A data race bug.

to use a new term ‘operation block’, which delivers the high level meaning clearly because the existing terms defined in specific contexts are not suitable for explaining different techniques.

- **Data association** (S_{DA} of Definition 1 in Section 3.2)

Variables of a data structure are often correlated to each other and may have implicit or explicit invariants. Thus, concurrent accesses to such variables should be synchronized and coordinated to satisfy the invariants/relations. Some race bug detection techniques receive the relation of correlated variables as a requirement specification and check the ‘correctness’ of access operations to the variables. In other words, they check if these operations are coordinated correctly to satisfy the invariants/relations over correlated variables.

Data association specifications are obtained from the structure of composite data types (e.g. struct, object, array) [20, 21] or obtained from user-given specifications [6, 12, 13]. Some techniques implicitly infer data association specifications from access patterns to variables [2, 8, 18, 19, 22]. Data association corresponds to the terms ‘multi-variable correlation’ [22], ‘atomic-set’ [6], ‘causal dependency between two variables’ [7] or ‘view’ [18] that occur in the literature.

We classify race bugs into four classes depending on the usage of operation block specifications and/or data association specifications in the race bug definition; race bug detection techniques are then classified according to their target race bug class. The high-level overviews of the four race bug classes are as follows (the formal race bug definitions are described in Sections 4–7).

- **Data race bug** (Section 4)

A data race bug is defined as two operations of different threads that concurrently read and write (or both write) data on the same memory location without synchronization. Figure 2 illustrates an example of a data race bug: operation₁₁ of Thread₁ and operation₂₁ of Thread₂ can update the same memory location m without synchronization. A data race bug coincides with ‘data race’ in Netzer *et al.* [23].

- **Block race bug** (Section 5)

A block race bug is defined as three operations p , p' and q , where p and p' of the same thread belong to an operation block, and q of another thread interferes the intended data-flow between p and p' (this intention is specified by an operation block). A block race bug can be harmful because this interfered data flow may result in unintended programme behaviours and raise an error. Figure 3 illustrates an example of a block race bug; operation₂₁ of Thread₂ interferes the intended data flow on m between operation₁₁ to operation₁₂ of Thread₁. Block race bug is called as ‘AI-invariant violation’ [3] or ‘atomicity violation’ [9, 10, 24] in the literature.[¶]

[¶]Some papers use the term ‘atomicity violation’ to refer to multi-data race bug (e.g. [2]).

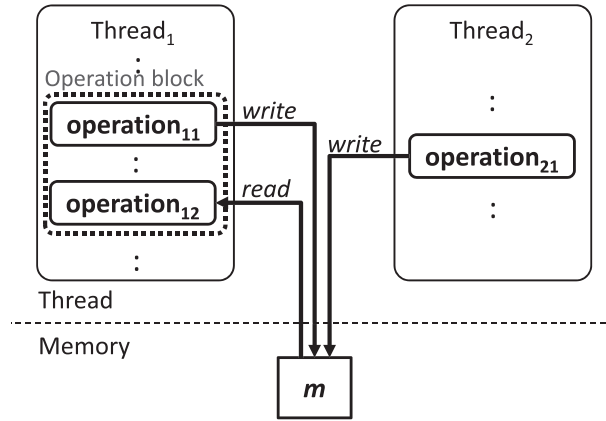


Figure 3. A block race bug.

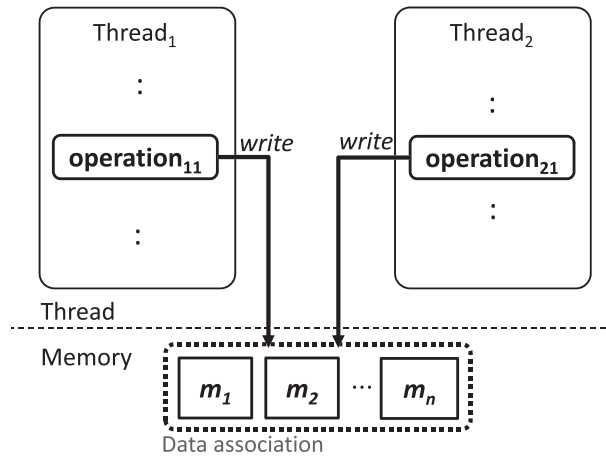


Figure 4. A multi-data race bug.

- *Multi-data race bug* (Section 6)

A multi-data race bug is defined as two operations of two different threads that manipulate data associated memory locations without proper synchronization. The multi-data race bug definition extends the data race bug definition to check conditions on data association relations. Figure 4 shows an example of a multi-data race bug. Because the accesses to the two data associated memory locations (e.g. m_1 and m_2) are not synchronized, the data structure may become inconsistent. The class of multi-data race bugs covers ‘multi-variable race’ [22] and ‘object race’ [21] in the literature.

- *Multi-data block race bug* (Section 7)

A multi-data block race bug is defined as three operations such that two of these are executed by a thread in the same operation block, the other operation is executed by another thread, and the three operations manipulate memory locations with data association relations. The multi-data block race bug definition extends the block race bug definition with an additional data association check. Figure 5 describes an example of a multi-data block race bug. As a result of interleaved executions of the three operations, the execution may have unintended combinations of data flows on the data associated variables. The multi-data block race bug definition corresponds to ‘high-level data race’ [18], ‘atomic-set serializability violation’ [6, 25], ‘multi-variable atomicity violation’ [22] or ‘atomicity violation’ [2] in the literature.

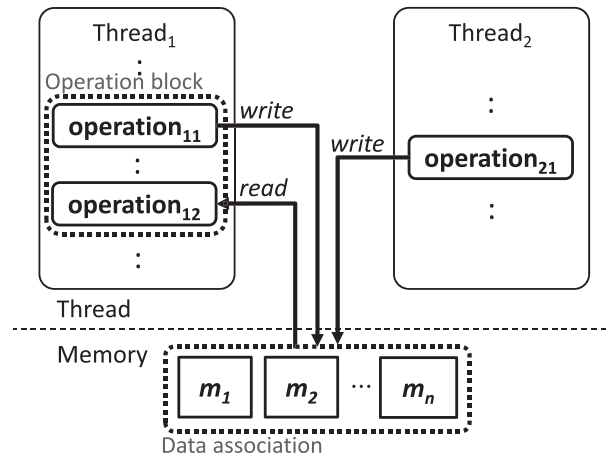


Figure 5. A multi-data block race bug.

3. EXECUTION MODEL OF MULTITHREADED PROGRAMMES

In this section, we formally define an execution model to represent the programme executions.

3.1. Overview

A race bug detection technique constructs and analyses an execution model that represents the concurrent executions of a target multithreaded programme. Dynamic race bug detection techniques execute the target programme to construct an execution model of the programme based on the execution traces. Static techniques construct a predictive execution model from the target programme code by analysing the control flow, data flow and synchronization effects of the programme. Most race bug detection techniques have their own definitions of execution models. To describe various definitions of the execution models used by race bug detection techniques, we define a general execution model that captures the essential aspects of concurrent executions without language technique or analysis technique-specific features.

Our execution model represents the executions of a multithreaded programme as *operations* and *synchronized order relations* of the operations. In the model, an operation represents an atomic action of the target programme execution (an operation is either read or write). A synchronized order relation represents the order of operations enforced by synchronization. In addition, our execution model represents *operation blocks* as a relation on operations and *data associations* as a relation on the variables of a target programme. These specifications are used to define the block race bugs, the multi-data race bugs and the multi-data block race bugs.

3.1.1. Examples of execution models. This section shows how two different race bug detection techniques can be compared clearly by using our execution model. Suppose that we have a target programme (Figure 6(a)) and an observed execution of the programme (Figure 6(b)). Suppose that we would like to compare FastTrack [26] and Eraser [1] both of which target data race bugs. To check if two operations can execute concurrently without synchronization, FastTrack utilizes *happens-before* relation and Eraser utilizes *lockset* algorithm.

Using our execution model definition, we show that FastTrack and Eraser may construct different execution models from the same code and observed execution (i.e. Figure 6). Figure 7(a) and 7(b) shows the execution models constructed from Figure 6 by FastTrack and Eraser, respectively. In the execution model, operations represent executions of instructions (i.e. reads and writes) which are not synchronizing instructions. An operation p_i denotes the execution of line number i in the example code. For example, the execution of line 1 (i.e. $x=1$) is represented by an operation $p_1 = (th1, write, x)$, which indicates a write operation executed by thread $th1$ on a variable x . The rest of the execution can be modelled in a similar way. Executions of synchronization instructions (e.g.

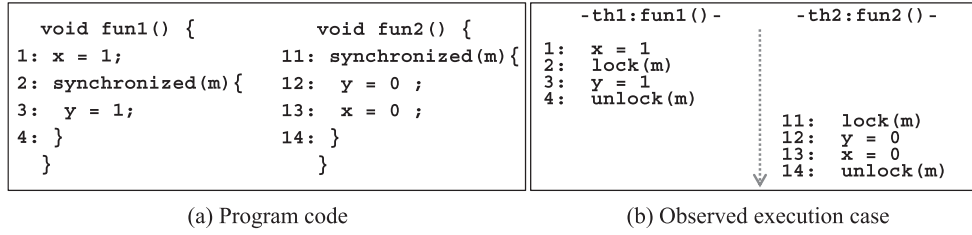


Figure 6. Example of a target programme code and an observed execution.

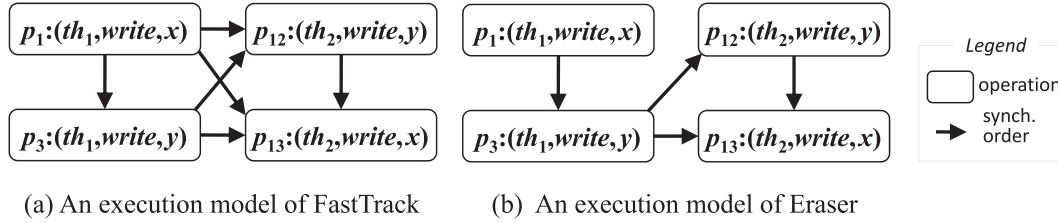


Figure 7. Examples of constructed execution models.

lines 1, 4, 11 and 14) are represented by the *synchronized order* relations. If there is a synchronized order between two operations p and q , it means that q always starts only after p finishes.

FastTrack uses the happens-before relation to build a synchronized order relation. In other words, FastTrack builds a synchronized order relation from the order of operations in the same thread (i.e. (p_1, p_3) and (p_{12}, p_{13})) and the order of operations enforced by a lock m (i.e. (p_3, p_{12}) , (p_3, p_{13})) in the observed execution and their transitive relations (i.e. (p_1, p_{12}) , (p_1, p_{13})). FastTrack builds an execution model with synchronized order relation $\rightarrow_{S_{FT}}$ as follows:

$$\rightarrow_{S_{FT}} = \{(p_1, p_3), (p_1, p_{12}), (p_1, p_{13}), (p_3, p_{12}), (p_3, p_{13}), (p_{12}, p_{13})\}.$$

In contrast, Eraser uses only mutual exclusion constraints induced by lock m and the operation order in each thread to generate synchronization order relations. Eraser generates a concurrent execution model with the synchronized order relation $\rightarrow_{S_{ER}}$ as follows:

$$\rightarrow_{S_{ER}} = \{(p_1, p_3), (p_{12}, p_{13}), (p_2, p_{13}), (p_3, p_{13})\}.$$

Thus, the data race bug detection results of FastTrack and Eraser can be different because their execution models of the same programme and its observed execution can be different as shown earlier. For the example in Figure 6, Eraser reports p_1 and p_{13} as a race bug because p_1 and p_{13} write to x and there is no synchronized order between them in $\rightarrow_{S_{ER}}$. In contrast, FastTrack does not report these two operations as a race bug because there is a synchronized order between these two operations in the execution model (i.e. $\rightarrow_{S_{FT}}$).

3.2. Formal definition of execution model

The proposed execution model aims to describe different execution models used by various race bug detection techniques in a uniform manner. For this purpose, the execution model in Definition 1 is *parametric* to race bug detection techniques. For example, the operation in the definition is not restricted to a particular programming language or analysis technique but general enough to represent all dynamic/static analysis techniques in this survey. In addition, we allow different definitions of *conflict* for different techniques (Table II).

Definition 1 (Execution model)

For a race bug detection technique and a target programme P with given synchronization requirement specifications (i.e. operation block specification and data association specification), an

execution model is defined as

$$\langle \Sigma, Thread, Mem, \rightarrow_S, conflict, S_{OB}, S_{DA} \rangle,$$

which consists of the following six elements:

- *A set of operations Σ*
 Σ is a set of operations (i.e. $\{p_1, p_2, \dots\}$), each of which models a unit of execution of P . An operation p has the following attributes whose concrete definitions depend on a race bug detection technique.
 - $thread(p) \in Thread$ indicates a thread that executes p .
 - $optr(p) \in \{\text{read}, \text{write}\}$ indicates a type of p , which is either data read or data write.
 - $operand(p) \in Mem$ indicates a memory location or variable manipulated by p (i.e. the operand of the p operation).
- *A set of threads $Thread$*
- *A set of memory locations Mem*
- *A synchronized order relation on operations $\rightarrow_S: \Sigma \times \Sigma$*
 $\rightarrow_S$ is a transitive closure of a binary relation on a set of operations $Sync: \Sigma \times \Sigma$ such that $(p, q) \in Sync$ if at least one of the following conditions holds:
 - (a) p and q are executed by the same thread and p is executed before q , or
 - (b) Explicit synchronization causes p to finish before q starts.

Different race bug detection techniques may generate $\rightarrow_S$ differently (Section 3.1.1). For example, there are techniques that construct $\rightarrow_S$ using happens-before relations enforced by lock operations in the observed execution (e.g. [2, 26, 27]). Other techniques may generate $\rightarrow_S$ using mutual exclusion relations enforced by lock operations in the observed execution (e.g. [1, 28, 29]).^{||}

- *A conflict relation $conflict: \Sigma \times \Sigma$*
 $(p, q) \in conflict$ (i.e. p and q conflict with each other) indicates that p and q can interfere each other and cause invalid behaviours. Formally speaking, $(p, q) \in conflict$ holds if the computational effect of p and q depends on the order in which they are executed. A conflict relation should be transitive and symmetric. Again, a concrete definition of $conflict$ may vary depending on the detection technique.
- *A set of operation blocks $S_{OB}: \Sigma \times \Sigma$*
 $(p, p') \in S_{OB}$ indicates that p and p' are executed by the same thread and p executes before p' in an operation block (i.e. $p \rightarrow_S p'$). $(p, p') \in S_{OB}$ indicates that execution from p to p' should not be interfered by another thread.
- *A relation of data association $S_{DA}: Mem \times Mem$*
 S_{DA} indicates a relation on shared variables as described in Section 2. $(v_1, v_2) \in S_{DA}$ indicates that v_1 and v_2 are parts of a composite data structure, and update operations on v_1 and v_2 should be coordinated to satisfy the data structure invariants. Note that S_{DA} is reflexive (i.e. $\forall v \in Mem. (v, v) \in S_{DA}$); S_{DA} may or may not be transitive/symmetric depending on the technique (Tables III and IV).

3.3. Execution model generation

Many race bug detection techniques check the programme behaviour against race bug definitions as soon as a part of the execution model is constructed (i.e. on-the-fly) for efficiency. In this paper,

^{||}Note that our execution model represents the *effect* of synchronization with $\rightarrow_S$ not the synchronization itself. Thus, this execution model is general enough to describe various race bug detection techniques with different synchronization mechanisms.

these two steps are distinguished to describe various techniques in a uniform manner. A total 43 race bug detection techniques are classified (Sections 4–7) according to how they generate an execution model.

- **Dynamic techniques** (34 out of the 43 techniques)
These techniques instrument the target programme to extract concrete values of programme executions at runtime, such as thread identifiers, memory addresses, temporal ordering of operations including synchronization and so on. These techniques can generate an execution model that does not contain infeasible behaviours (i.e. a bug detected by these techniques is always real). However, the generated execution model may miss buggy executions and miss race bugs as a result (i.e. false negatives). Most of the techniques (34 out of 43) in this survey belong to this class [1–4, 6, 8–14, 17, 20–22, 24, 26–28, 30–41]. For more detail on how each technique constructs an execution model, see Tables I–IV.
- **Static techniques** (Seven out of 43 techniques)
These techniques utilize static analysis methods such as alias analysis [42], data flow analysis [16], dependency analysis [7] and type systems [43] to obtain the information necessary to construct an execution model. These techniques can generate execution models representing a large scope of possible programme executions. However, the accuracy of the resulting execution models is often low because of the imprecision inherent to static analysis methods. Thus, these techniques may raise false positives. In this survey, seven out of 43 techniques belong to this class [7, 16, 19, 29, 42–44].
- **Hybrid (dynamic + static) techniques** (Two out of the 43 techniques)
HAVE [15] constructs execution models by combining runtime information obtained by dynamic analysis and code information obtained from a static analysis. McPatom [5] also combines dynamic execution information and source code information to generate execution models by considering programme semantics.

Most race bug detection techniques generate multiple execution models to check as many thread scheduling scenarios as possible. To generate multiple execution models from a single observed execution (i.e. to predict alternative scheduling scenarios [45]), dynamic techniques mutate orders of operations while satisfying the relations/constraints enforced by synchronization code. Static techniques analyse a target programme including synchronization code to build execution models that represent various thread scheduling scenarios and satisfy the synchronization constraints. These synchronization constraints are used to build a synchronized order relation (i.e. $\rightarrow_S$ in Definition 1).

4. DATA RACE BUG DETECTION TECHNIQUES

In this section, *data race bug* is defined based on the formal execution model. The data race bug definition does not use the operation block specification nor data association specification, and it is the simplest class of race bugs presented in this paper. This section presents the formal definition of data race bug and then presents a survey of race bug detection techniques for data race bugs.

4.1. Bug definition

A data race bug is defined as two unsynchronized operations of two different threads that access the same shared variable, where at least one is a write operation. The data race bug definition in this paper coincides with the definition of ‘data race’ in Netzer *et al.* [23]. Many race bug detection techniques [1, 26–32, 34–39, 42–44] target to detect the data race bugs and implement various methods to detect them efficiently (Section 4.3). The definition of the data race bug using the formal execution model is as follows:

Definition 2 (Data race bug)

A programme has a data race bug if an execution model of the programme $\langle \Sigma, \text{Thread}, \text{Mem}, \rightarrow_S, \text{conflict}, S_{OB}, S_{DA} \rangle$ satisfies all of the following conditions:

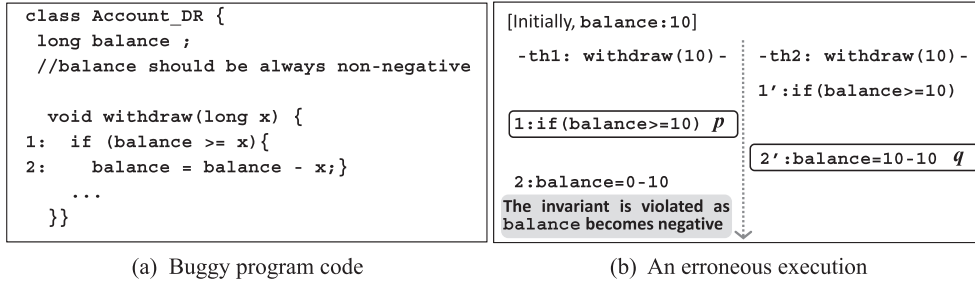


Figure 8. Data race bug example.

There exist two operations $p, q \in \Sigma$ such that

- DR1: $thread(p) \neq thread(q)$
- DR2: $operand(p) = operand(q)$
- DR3: $conflict(p, q)$
- DR4: $p \not\rightarrow_s q \wedge q \not\rightarrow_s p$

The data race bug definition indicates lack of synchronization on accesses to a shared variable. The DR1 condition checks if p and q are executed by different threads. The DR2 and DR3 conditions together check if p and q may interfere with each other. Note that various techniques have different definitions of $conflict()$ (see the fourth column of Table I). Finally, the DR4 condition checks if p and q can run concurrently without synchronization. In other words, for two operations p and q constituting a data race bug, no synchronization is enforced between p and q (i.e. $p \rightarrow_s q$ or $q \rightarrow_s p$).

Two operations that match the data race bug definition can be harmful because unsynchronized accesses to a shared variable may induce unintended behaviours. Especially, with weak memory models and multicore CPUs, executions depend on runtime status of CPU and shared variable accesses and often produce counterintuitive results [46, 47]. Enforcing synchronization on all accesses to a shared variable makes the accesses predictable because the access results are consistently determined by an access order. In addition, data race bugs may be harmful even with the sequentially consistent memory model because a data race bug is caused by unsynchronized concurrent operations on the shared variable, which may not have been intended by developers [48].

4.2. Data race bug example

Figure 8 shows an example of a data race bug in a buggy programme code and an erroneous execution scenario. The example is a bank account programme that includes method `withdraw(long x)`. Although `withdraw(long x)` should not withdraw x if x is larger than `balance` (line 1) to keep `bal` non-negative, the programme allows negative balance due to the data race bug explained as follows.

Suppose that, as shown in Figure 8(b), there are two threads (`th1` and `th2`) running the bank account programme and `balance` is 10 initially. Also, suppose that a data race bug detection technique similar to Eraser [1] is used to generate an execution model from the execution in Figure 8(b) with the following elements:

- $\Sigma = \{p_1, p_2, p_1', p_2', \dots\}$
- $Thread = \{th1, th2\}$
- $operand(p) = operand(q) \in Mem$
- $\rightarrow_s = \{(p_1, p_2), (p_1', p_2'), \dots\}$
- $(p, q) \in conflict$ if either $optr(p)$ or $optr(q)$ is write
- $S_{OB} = \emptyset$
- $S_{DA} = \emptyset$

Operation p (i.e. p_1) of $th1$ and operation q (i.e. p_2') of $th2$ (satisfying DR1) access the same shared variable `balance` (satisfying DR2). In addition, the technique considers that p and q *conflict* with each other (see the fourth column of the fifth row of Table I) because $optr(p)$ is `read` and $optr(q)$ is `write` (satisfying DR3). Furthermore, p and q can run concurrently without synchronization (i.e. $p_1 \not\rightarrow_S p_2'$ and $p_2' \not\rightarrow_S p_1$ (satisfying DR4)). Because the given execution satisfies all conditions of the data race bug definition, the technique reports a data race bug. This data race bug actually causes an error by assigning -10 to `balance` (line 2 in $th1$), which violates the non-negative invariant on `balance`.

4.3. Data race bug detection techniques

Table I summarizes 17 techniques for detecting the data race bugs. Each column describes how these techniques check each condition in Definition 2. The four columns (labelled DR1 through DR4 in the top row) are as follows:

- The second column describes mechanisms/methods to obtain thread information of an operation p to check the DR1 condition. The techniques with ‘runtime thread identifier’ [1, 26–28, 30–32, 34–39] utilize thread identifiers observed in monitored runtime traces to check if two operations are executed by different threads. The techniques with ‘static approximation’ [29, 42–44] obtain thread identifiers statically, which may produce imprecise results due to the lack of thread sensitivity in static analyses. To obtain a thread identifier, some techniques with ‘static approximation’ try to identify code segments that cannot be executed concurrently, such as interrupt handlers.
- The third column describes the methods to check whether or not p and q may access the same memory location (DR2). The techniques with ‘code analysis’ [29, 42–44] perform alias analyses on the target programme code to check if two operations may access the same memory location in any execution. For this purpose, Chord [42] exploits data flow analyses, and RELAY [44] utilizes symbolic executions. RccJava [43] exploits a type system to group operations that may access to the same object.

The techniques with ‘runtime memory address’ [1, 26–28, 30–32, 34–39] use memory locations observed in runtime traces as operands to check if $operand(p)$ is the same as $operand(q)$. Many techniques try to reduce the runtime overhead involved in monitoring all operations. O’Callahan *et al.* [35] leverage static analysis to select statements that may produce data race bugs and then instrument only those statements to be monitored at runtime. SOS [38] statically finds read-only variables (called *stationary variables*) and then does not monitor accesses to them to save the runtime monitoring cost. Another approach to reduce the runtime monitoring overhead is selecting a subset of memory locations that are likely to be involved in data race bugs to monitor their accesses. LiteRace [34] and ThreadSanitizer [39] utilize the *cold region* hypothesis, which states that frequently tested code segments are less likely to have data race bugs and exclude frequently executed statements from monitoring. As an alternative approach, RACEZ [37] tries to reduce the monitoring cost by monitoring sampled target memory accesses at the cost of missing data race bugs.

- The fourth column describes the methods to check if two operations *conflict* with each other (DR3). All data race bug detection techniques in Table I consider a combination of a read operation and a write operation (read–write or write–read) or a combination of two write operations (write–write) as conflict. RccJava [43] additionally considers the case of two read operations as conflict because the type system of RccJava requires that every operation on a shared variable should be guarded by a lock regardless of the access type.
- The fifth column explains which synchronization mechanisms each technique utilizes to construct the synchronized order relation $\rightarrow_S$ of an execution model to check DR4. All techniques in Table I utilize observed lock operations (denoted by ‘lock’). The techniques with ‘message send/receive’ [34–36] use message send-and-receive style synchronization as well. The techniques with ‘thread create/join’ [26, 28, 31, 36, 38] additionally use thread operations. LiteRace [34] utilizes atomic operations, and GUARD [32] utilizes barriers in addition to lock operations.

5. BLOCK RACE BUG DETECTION TECHNIQUES

Block race bug detection techniques aim to detect race bugs that are caused by unintended concurrent accesses to a shared variable by an operation block and an operation. More specifically, the block race bug detection techniques detect three operations as a block race bug if two operations access the same variable in the same operation block, and the other operation in another thread can access the same variable concurrently to interfere the operation block execution.

5.1. Block race bug definition

The block race bug definition is defined in terms of the following three operations: two operations p and p' in an operation block and one operation q that can run concurrently with the two operations. The block race bug definition describes that p and p' in the operation block should not be interfered by the other concurrent operation q for accessing the shared variable m . In the literature, block race bugs are sometimes called as ‘atomicity violation’ [9, 10, 24] or ‘AI-invariant violation’ [3]. Note that the term ‘atomicity violation’ also refers to a different class of race bugs (i.e. multi-data block race bugs in Section 7).

The formal definition of the *block race bug* is as follows.

Definition 3 (Block race bug)

A programme has a block race bug if an execution model of the programme $\langle \Sigma, Thread, Mem, \rightarrow_S, conflict, S_{OB}, S_{DA} \rangle$ satisfies all of the following conditions.

There exist three operations p, p' and $q \in \Sigma$ such that

- BR1: $thread(p) = thread(p')$
- BR2: $(p, p') \in S_{OB}$
- BR3: $operand(p) = operand(p')$
- BR4: $thread(p) \neq thread(q)$
- BR5: $operand(p) = operand(q)$
- BR6: $conflict(p, q) \wedge conflict(q, p')$
- BR7: $q \not\rightarrow_S p \wedge p' \not\rightarrow_S q$

The BR1–BR3 conditions check if two operations p and p' of the same thread (BR1) access the same variable (BR3) in an operation block (BR2). Note that for two operations p and p' that satisfy BR2, p is executed before p' in the operation block (i.e. $p \rightarrow_S p'$).

The BR4–BR7 conditions check if operation q can run concurrently with p and p' and access the same variable. The BR4 and BR7 conditions check if q is executed by a different thread from the thread of p and p' (BR4), and q can be executed while a thread executes p and p' in an operation block (i.e. if q can execute between p and p') (BR7). Note that q does not have to belong to any operation block. The BR5–BR6 conditions check if p and q access the same variable and thus potentially interfere each other (similarly for q and p').

Note that the BR4–BR7 conditions are similar to the DR1–DR4 conditions for the data race bug definition, whereas BR4–BR7 have additional constraints for the relation between q and p' . In Section 8.1, a detailed comparison of the block race bugs with the data race bugs is presented.

5.2. Block race bug example

Figure 9 shows an example of a programme with a block race bug and an erroneous execution due to the block race bug. Note that the programme should keep *balance* non-negative. Lines 4–8 of method `void withdraw(long x)` are specified as an atomic region, thus the operations that execute lines 4–8 are defined as an operation block. Suppose that there are two threads (`th1` and `th2`) running the bank account programme in Figure 9, and *balance* is initially 10.

Suppose that a block race bug detection technique such as PENELOPE [4] (see the 11th row of Table II) is applied in Figure 9. Operations p and p' are executed by the `th1` thread (satisfying BR1)

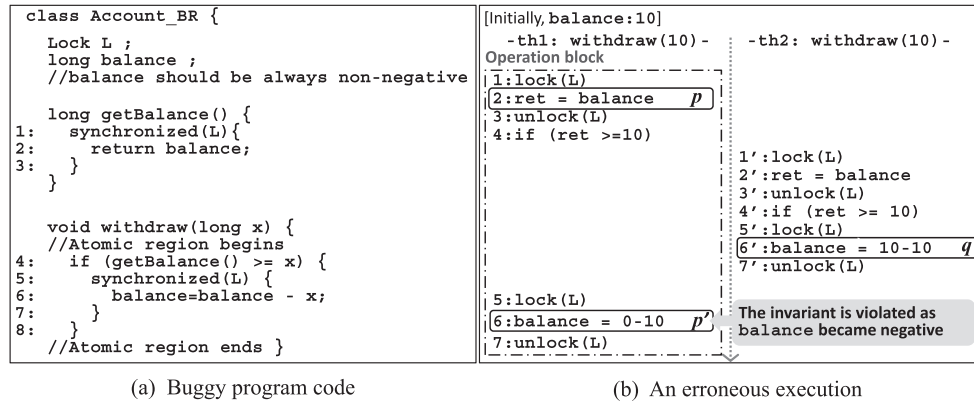


Figure 9. Block race bug example.

in the same operation block (satisfying BR2) and access the same variable balance (satisfying BR3) (Figure 9(b)).

The th2 thread executes *q* (satisfying BR4) that accesses the same variable balance (satisfying BR5). Operations *p* and *q* conflict with each other and operations *q* and *p'* have a conflict as well (satisfying BR6) because *optr(p)* is read, whereas *optr(q)* is write, and *optr(q)* is write, whereas *optr(p')* is write (see the fourth column of the 11th row of Table II). Finally, *q* occurs between *p* and *p'* (satisfying BR7). Therefore, the execution model for the given execution satisfies all conditions of the block race bug definition, and thus, the technique detects a block race bug. This execution actually results in an error by assigning -10 to balance (line 6 in th1), which violates the intention of the programme. The error occurs because the condition check on balance (line 4) and the following update of balance (line 6) are not guarded by the same synchronized block, which allows a concurrent update of balance (line 6) by the other thread.

5.3. Block race bug detection techniques

Table II summarizes the 10 block race bug detection techniques [3–5, 9–11, 24, 40, 41]. ** The four columns of the table are as follows:

- The second column describes the methods to check whether two operations are executed by the same thread or different threads (BR1 and BR4). All block race bug detection techniques in this survey utilize thread identifier information observed in actual traces to check the BR1 and BR4 conditions.
- The third column shows the methods to check if two operations access the same variable (BR3 and BR5). In this survey, all techniques compare concrete memory locations observed in actual traces.
- The fourth column shows the criteria to determine if two operations executed by different threads conflict with each other (BR6). The three techniques [4, 5, 41] consider that two operations conflict with each other when at least one operation is a write operation, which is denoted as ‘read–write, write–read, write–write’ in the table. The other seven techniques [3, 9–11, 24, 40] consider *p* and *q* (or *p'* and *q*) as conflicting with each other if they are ‘read–write, write–read, write–write’ on a variable. These techniques consider *p* and *q* as not conflicting if they write to a variable *m* and the operation block of *p* has another operation that overwrites *m* before any read operation (denoted by write–write). The techniques with ‘write–write’ conflict follow the notion of *conflict serializability*. The techniques with ‘write–write’ conflict follow the notion of *view serializability*.^{††}

** Our survey does not include Atomizer [49] because Atomizer checks locking patterns rather than data access patterns.

†† An in-depth comparison of the algorithmic complexity and bug detection precision for these two notions is found in Qadeer *et al.* [50].

- The last column describes the synchronization mechanisms to construct the synchronized order relation $\rightarrow_S$ to check BR7. For this purpose, six techniques [4, 5, 9, 10, 14, 24] track lock operations (denoted by ‘lock’). The block-based algorithm [14] additionally uses message send/receive operations, and CTrigger [24] and McPatom [5] consider other synchronization mechanisms including wait-and-notify, thread creation/join and barrier to generate more accurate execution models. The other four techniques [3, 11, 40, 41] approximately check BR7 based on the observed temporal orders of operations (denoted by ‘timestamp’) without considering explicit synchronization operations (i.e. check if starting time instances of p , q and p' are in sequence). These four techniques may miss a block race bug when operation q happens to be executed before p or after p' .

6. MULTI-DATA RACE BUG DETECTION TECHNIQUES

The multi-data race bug detection techniques target a race bug that inconsistently updates two associated variables specified in the data association specification S_{DA} . This type of race bugs is characterized by unsynchronized concurrent accesses to the associated variables.

6.1. Multi-data race bug definition

Definition 4 (Multi-data race bug)

A target programme has a multi-data race bug if an execution model of the programme $\langle \Sigma, \text{Thread}, \text{Mem}, \rightarrow_S, \text{conflict}, S_{OB}, S_{DA} \rangle$ satisfies all of the following conditions.

There exist two operations $p, q \in \Sigma$ such that

- MR1: $\text{thread}(p) \neq \text{thread}(q)$
- MR2: $(\text{operand}(p), \text{operand}(q)) \in S_{DA}$
- MR3: $\text{conflict}(p, q)$
- MR4: $p \not\rightarrow_S q \wedge q \not\rightarrow_S p$

The MR1, MR3 and MR4 conditions are the same as the DR1, DR3 and DR4 conditions of the data race bug, respectively. The MR2 condition utilizes data association S_{DA} to check if p and q access associated variables. Section 8.2 presents a detailed comparison of the multi-data race bugs and the data race bugs.

6.2. Multi-data race bug example

Figure 10 describes an example of a programme containing a multi-data race bug and an erroneous execution caused by the bug. In the example, *balance* and *debt* represent the amount of balance and the amount of debt in a bank account, respectively. The programme has an invariant that *debt* is zero when *balance* is positive (i.e. $(\text{balance} > 0) \rightarrow (\text{debt} == 0)$) and *balance* is zero

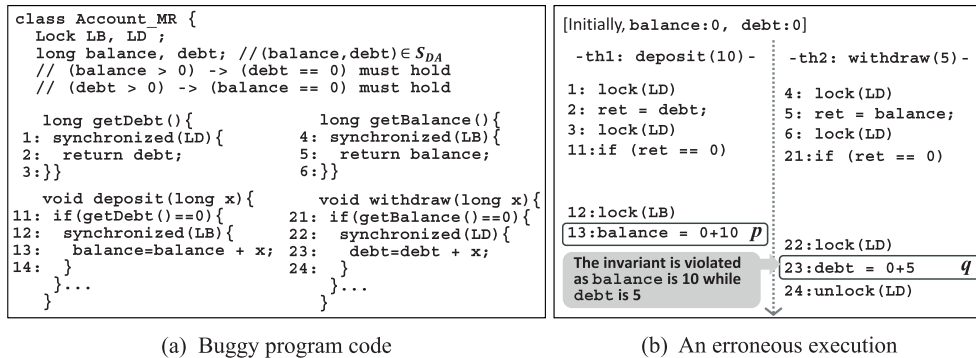


Figure 10. Multi-data race bug example.

when *debt* is positive (i.e. $(\text{debt} > 0) \rightarrow (\text{balance} == 0)$). Therefore, *balance* and *debt* are associated and should be updated consistently to satisfy the invariant. The `deposit(long x)` method increases *balance* by *x* when *debt* is zero (lines 11–14). The `withdraw(long x)` method increases *debt* by *x* when *balance* is zero (lines 21–24).

Suppose that a multi-data race bug detection technique similar to the object race detection technique [21] is used (see the fourth row of Table III). Suppose that there are two threads, *th1* and *th2*, running the bank account programme where *balance* is zero and *debt* is 10 initially. Also, suppose that *balance* and *debt* are associated to satisfy the invariant. Operation *p* on *th1* and operation *q* on *th2* (satisfying MR1) access *balance* and *debt*, respectively, (satisfying MR2 as $(\text{balance}, \text{debt}) \in S_{DA}$). In addition, *p* and *q* conflict with each other, because $\text{optr}(p)$ is write and $\text{optr}(q)$ is also write (satisfying MR3). Furthermore, *p* and *q* run concurrently without synchronization (satisfying MR4).

Therefore, the given execution scenario satisfies all conditions of the multi-data race bug definition, and thus, the multi-data race is detected. This execution actually causes an error, because *balance* becomes 10 and *debt* becomes 5, which violates the invariant. In other words, the interleaved execution updates associated variables inconsistently because *p* and *q* are protected by different locks.

6.3. Multi-data race bug detection techniques

Table III gives an overview of the multi-data race bug detection techniques. The columns represent the following:

- The second column shows that all three multi-data race bug detection techniques in this survey [20–22] use runtime thread identifiers observed in actual traces to check if two operations are executed by the same thread or not (for checking MR1).
- The third to sixth columns (MR2) represent the type and characteristics of the data association relation for each technique. Data association for a target execution model is specified as a relation over two variables. The third to fifth columns show whether or not the data association relation is reflexive (i.e. $\forall a \in \text{Mem.}(a, a) \in S_{DA}$), symmetric (i.e. $\forall a, b \in \text{Mem.}(a, b) \in S_{DA} \rightarrow (b, a) \in S_{DA}$) or transitive (i.e. $\forall a, b, c \in \text{Mem.}(a, b) \in S_{DA} \wedge (b, c) \in S_{DA} \rightarrow (a, c) \in S_{DA}$), respectively. The sixth column ('Type') shows the type of the data association relation used by the technique. MultiRace [20] and the object data race detection technique [21] construct data association relation as a set of two memory locations. MUVE-Eraser [22] defines a data association as an ordered pair of variables (i.e. not symmetric) with conditions on the access types to the variables (i.e. read or write).

In addition, all three techniques in this category utilize memory locations observed in actual traces to obtain operand information and check MR2.

- The seventh column shows that all three techniques in this class consider two operations as conflicting if they access the associated variables and at least one operation is a write operation (MR3).
- The last column represents that all three techniques track lock operations to build a synchronized order relation $\rightarrow_S$ to check MR4. MultiRace [20] additionally considers thread creation and join.

The methods that these techniques employ to obtain data association are as follows. MultiRace [20] aggregates continuous shared memory locations allocated for a composite data structure into one cluster and considers any two memory locations in the cluster as associated. Consequently, the data association relation becomes an equivalence relation (i.e. transitive, symmetric and reflexive). Similarly, object data race detection technique [21] considers members/fields of an object as associated data. These two techniques obtain the data association specification from data structures declared in a target programme. MUVE [22] utilizes data association specification inferred by a heuristic that considers two variables as associated if they are frequently accessed in close distance.

7. MULTI-DATA BLOCK RACE BUG DETECTION TECHNIQUES

This section presents the definition of *multi-data block race bug* and 13 techniques that target the multi-data block race bug. A multi-data block race bug consists of two operations within an operation block and another operation that runs concurrently with the two operations that access associated variables in unsynchronized manner. The race bug detection techniques in this category detect multi-data block race bugs that violate combined specifications of operation blocks and data associations.

7.1. Multi-data block race bug definition

Definition 5 (Multi-data block race bug)

A programme has a multi-data block race bug if an execution model of the programme $\langle \Sigma, \text{Thread}, \text{Mem}, \rightarrow_S, \text{conflict}, S_{OB}, S_{DA} \rangle$ satisfies all of the following conditions.

There exist three operations p, p' and $q \in \Sigma$ such that

MBR1: $\text{thread}(p) = \text{thread}(p')$

MBR2: $(p, p') \in S_{OB}$

MBR3: $(\text{operand}(p), \text{operand}(p')) \in S_{DA}$

MBR4: $\text{thread}(p) \neq \text{thread}(q)$

MBR5: $(\text{operand}(p), \text{operand}(q)) \in S_{DA} \wedge (\text{operand}(q), \text{operand}(p')) \in S_{DA}$

MBR6: $\text{conflict}(p, q) \wedge \text{conflict}(q, p')$

MBR7: $q \not\rightarrow_S p \wedge p' \not\rightarrow_S q$

The MBR1–MBR3 conditions specify that operations p and p' should be executed by the same thread (MBR1) within the same operation block (MBR2), and the $\text{operand}(p)$ and $\text{operand}(p')$ are data associated (MBR3). The MBR4–MBR7 conditions check if there exists an operation that can be executed by another thread (MBR4), between p and p' (MBR7), and the operation can interfere with the data association of p and p' (MBR5 and MBR7).

There exist techniques that have additional condition for MBR5. The five techniques [2, 8, 15, 17, 33] additionally check if the operands of p and q (or the operands of q and p') should be the same. Other two techniques [18, 19] detect three operations as a multi-data race bug only if at least two associated variables are involved with the three operations.

The multi-data block race bug definition is an extension of the block race bug definition (Definition 3) to check additional conditions on data association relations, in a way similar to the multi-data race bug definition (Definition 4). The MBR1, MBR2, MBR4, MBR6 and MBR7 conditions are identical to the BR1, BR2, BR4, BR6 and BR7 conditions in the block race bug definition. The MBR3 and MBR6 conditions are adapted from MR2 to check if variables accessed by two operations are associated. Section 8.3 presents a detailed comparison of block race bug and multi-data block race definitions. Similarly, the comparison of the multi-data race bug definition with the multi-data block race bugs is presented in Section 8.4.

7.2. Multi-data block race bug example

Figure 11 shows an example of a multi-data block race bug. The concurrent execution causes an error because operations p and p' that access two associated variables (*balance* and *debt*) should be synchronized as a whole but separately. Thus, because of the intervening q of *th2*, the values of *balance* and *debt* are updated inconsistently and violate the invariant.

7.3. Multi-data block race detection techniques

Table IV is an overview of 13 multi-data block race detection techniques.

- The second column shows that the 10 techniques with ‘runtime thread identifier’ use the information of thread identifiers observed in actual traces to check the MBR1 condition [2, 6, 8, 12,

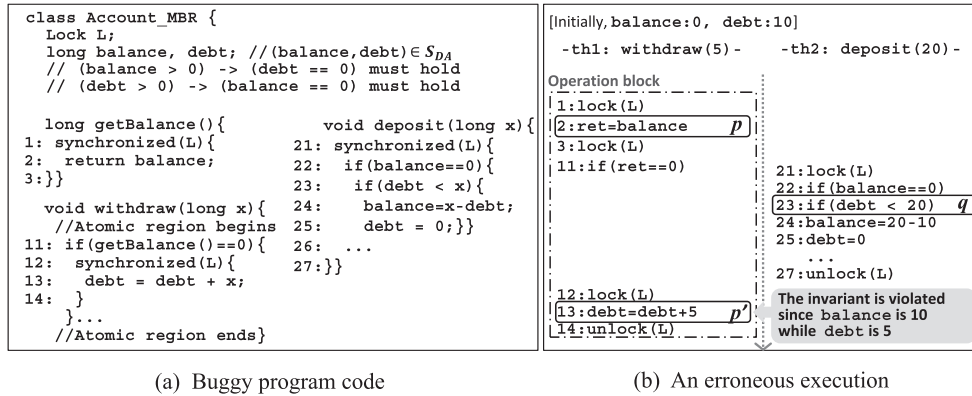


Figure 11. Multi-data block race bug example.

13, 15, 17, 18, 22, 33]. The other three techniques [7, 16, 19] consider that two operations of two public methods/functions can be executed by different threads.

- The third to sixth columns show the properties of data association relation S_{DA} in each technique (for checking MBR3 and MBR5). The data association relations used by the three techniques [6, 12, 13] are reflexive, transitive and symmetric. Thus, the data association relation of an execution model can be represented as a set of partitions of memory locations. For example, an *atomic set* [6, 25] defines a set of member fields of an object as associated. The sixth column shows that the three techniques [7, 19, 22] generate the data association relation by specifying two variables with access types (read or write).
- The seventh column represents the method to obtain operand information for each technique to check the MBR3 and MBR5 conditions. The eight techniques with ‘runtime mem.’ [2, 6, 8, 12, 13, 17, 22, 33] use runtime memory addresses observed in actual traces. The four techniques with ‘code analysis’ use static analysis to estimate operand information. HAVE [15] utilizes a hybrid method that use both runtime trace and code analysis result.
- The eighth column shows that the four techniques [7, 16, 18, 19] consider that two operations accessing associated variables conflict with each other (MBR6) regardless of their types. Other eight techniques [2, 6, 8, 12, 13, 15, 17, 33] consider two operations as conflicting, when at least one operation is a write operation. MUVI-AVIO [22], an extension of AVIO [3], does not consider two write operations p and q as conflicting, when another write operation in the same operation block as p overwrites the variable before any read (denoted by write–write’).
- The last column shows that the eight techniques with ‘lock’ [2, 7, 8, 15–19] obtain synchronized order relations by tracking lock synchronization (MBR7). In addition to tracking lock ordering, COPPER [8] considers thread creation/join and wait/notify. The commit node algorithm [17] and HAVE consider the execution order induced by message send/receive to build the synchronized order $\rightarrow_S$. The five techniques with ‘timestamp’ [6, 12, 13, 22, 33] use the observed temporal ordering of operations to generate the synchronization order relation without considering explicit synchronization instructions (i.e. $p \rightarrow_S q$ if $timestamp(p) < timestamp(q)$). Because these methods over-approximate the synchronization relation, the five techniques may miss existing multi-data block race bugs.

The multi-data block race bug detection techniques use different methods to obtain data association specification. First, the three techniques [6, 12, 13] ask users to concretely specify the data association specification. The other techniques use programme analysis or heuristics to infer data association relations from the target programme code or observed runtime traces. Dias *et al.* [7] analyse the target programme code to identify control dependency and data dependency relation in shared variable accesses and then consider two variables with data dependency or control dependency as associated. MUVI-AVIO [22] infers likely data association relations by analysing statistics on the variable accesses in the target programme code. The other techniques [2, 7, 8, 15–19, 33] consider two variables accessed in an operation block as data associated. The block-local atomicity

technique [16] defines two variables x and y as associated when the two variables are accessed in the same method and there exists a local variable that is assigned by the value of x and then used to define y .

8. RELATIONS BETWEEN RACE BUG CLASSES

Based on the formal definitions of the four classes of race bugs, it is possible to clarify the relation amongst the four classes of race bugs using the following notations:

- F_α denotes a conjunctive formula of all conditions of α race bug definition, where $\alpha \in \{\text{DR}, \text{BR}, \text{MR}, \text{MBR}\}$ represent classes of data race bugs, block race bugs, multi-data race bugs and multi-data block race bugs, respectively.
- f_c denotes a condition c of a race bug definition, where $c \in \{\text{DR1}, \text{DR2}, \text{DR3}, \text{DR4}\} \cup \{\text{BR1}, \text{BR2}, \text{BR3}, \text{BR4}, \text{BR5}, \text{BR6}, \text{BR7}\} \cup \{\text{MR1}, \text{MR2}, \text{MR3}, \text{MR4}\} \cup \{\text{MBR1}, \text{MBR2}, \text{MBR3}, \text{MBR4}, \text{MBR5}, \text{MBR6}, \text{MBR7}\}$
- ϕ_α denotes the set of execution models satisfying F_α . For every execution model σ , $\sigma \models F_\alpha$ if and only if $\sigma \in \phi_\alpha$. ϕ_α^C denotes the complement of the set ϕ_α . ϕ_α^C defines the set of execution models that satisfies $\neg F_\alpha$.

8.1. Data race bugs versus block race bugs

The set of execution models that satisfy the data race bug condition (i.e. ϕ_{DR}) overlaps with the set of execution models that satisfy the block race bug conditions (i.e. ϕ_{BR}). Figure 12 describes the relation of ϕ_{DR} and ϕ_{BR} , and the examples correspond to each region of the diagram. We can prove the relation by showing that $\phi_{DR} \not\subseteq \phi_{BR}$, $\phi_{DR} \cap \phi_{BR} \neq \emptyset$, and $\phi_{BR} \not\subseteq \phi_{DR}$.

Theorem 1

$$\phi_{DR} \not\subseteq \phi_{BR}$$

Proof

$\phi_{DR} \not\subseteq \phi_{BR}$ holds if and only if $\phi_{DR} \cap \phi_{BR}^C \neq \emptyset$. $\phi_{DR} \cap \phi_{BR}^C \neq \emptyset$ holds if and only if $F_{DR} \wedge \neg F_{BR}$ is satisfiable (i.e. there exists an execution model σ such that $\exists p, q, p' \in \Sigma. F_{DR} \wedge \neg F_{BR}$). We can rewrite the formula as follows: $F_{DR} \wedge \neg(f_{BR4} \wedge f_{BR5} \wedge f_{BR1} \wedge f_{BR2} \wedge f_{BR3} \wedge f_{BR6} \wedge f_{BR7})$. This formula is satisfiable if and only if there exists an execution model that satisfies $F_{DR} \wedge \neg(f_{BR1} \wedge f_{BR2} \wedge f_{BR3} \wedge f_{BR6} \wedge f_{BR7})$ because $F_{DR} \wedge \neg(f_{BR4} \wedge f_{BR5})$ is unsatisfiable as $f_{DR1} = f_{BR4}$ and $f_{DR2} = f_{BR2}$. Thus, $\phi_{DR} \not\subseteq \phi_{BR}$ holds if there exists an execution model that has two operations p and q , which satisfy the data race bug definition F_{DR} , whereas the execution model has no operation for p' in the operation block that contains p . $\square$

Figure 13 illustrates a programme with an execution scenario which has a data race bug but no block race bug. A data race bug detection technique such as Eraser [1] detects a data race bug for p and q because these read and write the variable `n_cache` without synchronization. However,

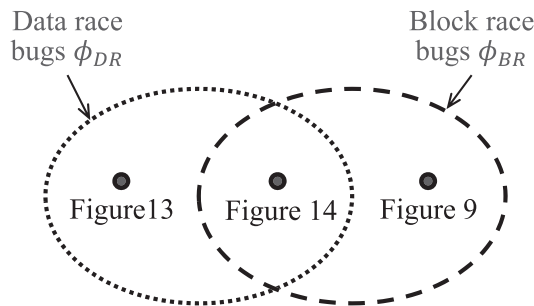


Figure 12. Relation between data race bugs and block race bugs.

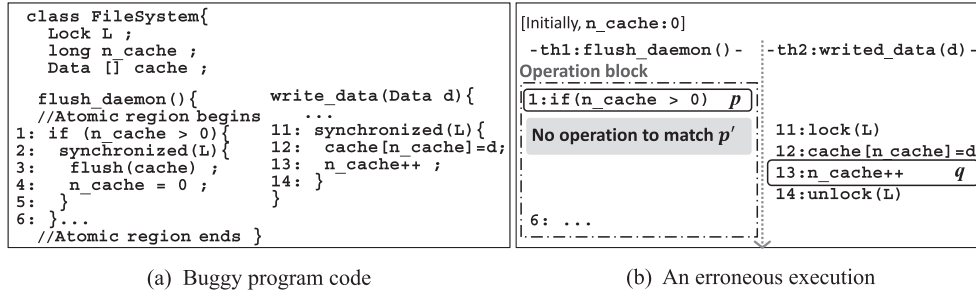


Figure 13. Example of a data race bug not detected by block race bug detectors.

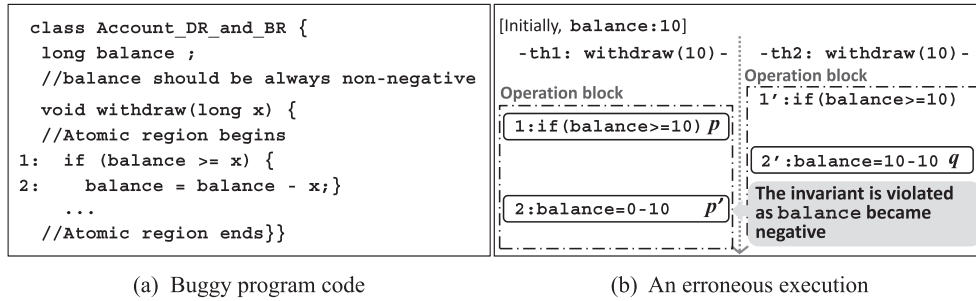


Figure 14. Example of a data race bug detected by block race bug detectors.

block race bug detection techniques such as AVIO [3] do not detect any block race bug for p and q because the execution model has no operation to match p' .

$$\phi_{DR} \cap \phi_{BR} \neq \emptyset$$

Proof

$\phi_{DR} \cap \phi_{BR} \neq \emptyset$ holds if and only if the following formula is satisfiable: $F_{DR} \wedge F_{BR}$. This formula is equivalent to $F_{DR} \wedge f_{BR1} \wedge f_{BR2} \wedge f_{BR3} \wedge f_{BR6} \wedge f_{BR7}$ as $DR1 = BR4$ and $DR2 = BR5$. Figure 14 describes an example that belongs to the intersection of ϕ_{DR} and ϕ_{BR} , which extends the example of Figure 8 by adding an operation block specification. A block race bug detection technique can detect operations p , p' and q as a race bug because p and p' belong to the same operation block of thread $th1$ (satisfying $BR1$ and $BR2$), all three operations access the same variable $balance$ (satisfying $BR3$), q and p' conflict with each other as both operations are writing (satisfying $BR6$) and q can run concurrently with p and p' (satisfying $BR7$). $\square$

$$\phi_{BR} \not\subseteq \phi_{DR}$$

Proof

$\phi_{BR} \not\subseteq \phi_{DR}$ holds if and only if $\phi_{BR} \cap \phi_{DR}^C \neq \emptyset$. The relation holds if and only if the following formula is satisfiable: $F_{BR} \wedge \neg F_{DR}$. The formula is equivalent to the formula $F_{BR} \wedge \neg f_{DR4}$ because $DR1 = BR4$, $DR2 = BR5$ and $BR6$ implies $DR3$ (i.e. $\forall \sigma. \sigma \models f_{BR6} \Rightarrow \sigma \models f_{DR3}$). Figure 9 shows an example, which has a block race bug but no data race bug. The data race bug detection techniques do not detect the data race bug in Figure 9 because p and q are synchronized (similarly, q and p'). $\square$

The practical implication of Theorem 1 is that it is a good idea to use data race bug detectors and block race bug detectors together as complements because data race bug detectors cannot detect all block race bugs and vice versa.

8.2. Data race bugs versus multi-data race bugs

Figure 15 shows that the class of data race bugs is a proper subset of the class of multi-data race bugs. We can prove this relationship using the race bug definitions.

Theorem 2

$$\phi_{DR} \subset \phi_{MR}$$

Proof

$\phi_{DR} \subseteq \phi_{MR}$ holds if and only if $\phi_{DR} \cap \phi_{MR}^C = \emptyset$. $\phi_{DR} \cap \phi_{MR}^C = \emptyset$ holds if and only if $F_{DR} \wedge \neg F_{MR}$ is unsatisfiable. $F_{DR} \wedge \neg F_{MR}$ is unsatisfiable if $f_{DR2} \wedge \neg f_{MR2}$ is unsatisfiable because $DR1 = MR1$, $DR3 = MR3$ and $DR4 = MR4$. $DR2$ implies $\exists v.(v, v) \in S_{DA} \wedge v = \text{operand}(p) = \text{operand}(q)$ (because $\forall v.(v, v) \in S_{DA}$). Note that if $\text{opernad}(p) = \text{operand}(q)$, $f_{MR2} = \exists v.(v, v) \in S_{DA} \wedge v = \text{operand}(p) = \text{operand}(q)$. Thus, $f_{DR2} \wedge \neg f_{MR2}$ is unsatisfiable and $F_{DR} \wedge \neg F_{MR}$ is unsatisfiable. Thus, $\phi_{DR} \cap \phi_{MR}^C = \emptyset$ and $\phi_{DR} \subseteq \phi_{MR}$ hold.

In addition, it is clear that there exists an execution model that satisfies the multi-data race bug definition but not the data race bug definition (for example, Figure 10). Thus, the class of data race bugs is a proper subset of the class of multi-data race bugs. $\square$

8.3. Block race bugs versus multi-data block race bugs

Figure 16 shows that the class of the block race bug is a proper subset of the class of the multi-data block race bug. We can prove this relation using the race bug definitions.

Theorem 3

$$\phi_{BR} \subset \phi_{MBR}$$

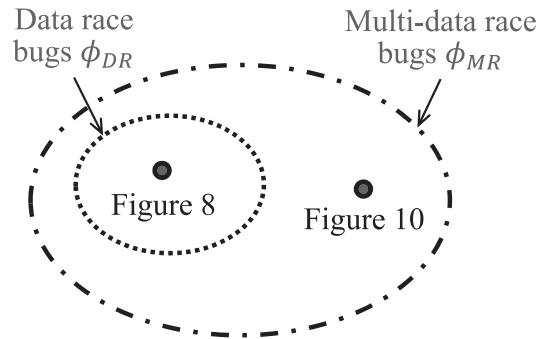


Figure 15. Relation between data race bugs and multi-data race bugs.

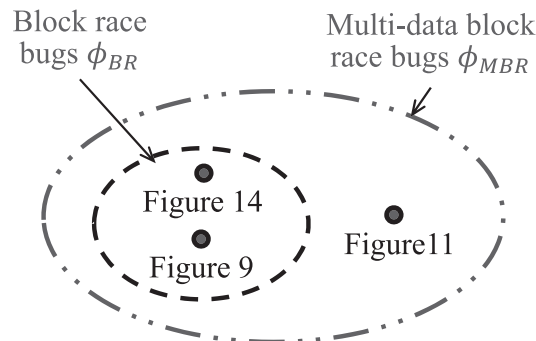


Figure 16. Relationship between block race bugs and multi-data block race bugs.

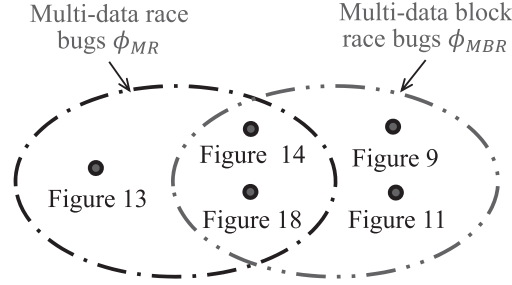


Figure 17. Relationship between multi-data race bugs and multi-data block race bugs.

Proof

$\phi_{BR} \subseteq \phi_{MBR}$ holds if and only if $\phi_{BR} \cap \phi_{MBR}^C = \emptyset$. $\phi_{BR} \cap \phi_{MBR}^C = \emptyset$ holds if and only if $F_{BR} \wedge \neg F_{MBR}$ is unsatisfiable. As $BR1 = MBR1$, $BR2 = MBR2$, $BR4 = MBR4$, $BR6 = MBR6$ and $BR7 = MBR7$, $F_{BR} \wedge \neg F_{MBR}$ is unsatisfiable if $(f_{BR3} \wedge f_{BR5} \wedge \neg f_{MBR3}) \vee (f_{BR3} \wedge f_{BR5} \wedge \neg f_{MBR5})$ is unsatisfiable. $BR3$ indicates $\exists v.(v, v) \in S_{DA} \wedge v = \text{operand}(p) = \text{operand}(q)$ (because $\forall v.(v, v) \in S_{DA}$). Note that if $\text{operand}(p) = \text{operand}(q)$, $f_{MBR3} = \exists v.(v, v) \in S_{DA} \wedge v = \text{operand}(p) = \text{operand}(q)$. Thus, $f_{BR3} \wedge \neg f_{MBR3}$ is unsatisfiable and $f_{BR5} \wedge \neg f_{MBR5}$ is also unsatisfiable for the similar reason. As a result, $F_{BR} \wedge \neg F_{MBR}$ is unsatisfiable.

In addition, it is clear that there exists an execution model that satisfies a multi-data block race bug definition but not the block race bug definition (for example, Figure 11). Thus, the class of block race bugs is a proper subset of the class of multi-data block race bugs. $\square$

8.4. Multi-data race bugs versus multi-data block race bugs

The class of the multi-data race bug ϕ_{MR} overlaps the class of the multi-data block race bug ϕ_{MBR} as shown in Figure 17. We can prove the relation of ϕ_{MR} and ϕ_{MBR} by showing that $\phi_{MR} \not\subseteq \phi_{MBR}$, $\phi_{MR} \cap \phi_{MBR} \neq \emptyset$ and $\phi_{MBR} \not\subseteq \phi_{MR}$.

Theorem 4

$$\phi_{MR} \not\subseteq \phi_{MBR}$$

Proof

$\phi_{MR} \not\subseteq \phi_{MBR}$ holds if and only if $F_{MR} \wedge \neg F_{MBR}$ is satisfiable. The formula is equivalent to $F_{MR} \wedge \neg(f_{MBR1} \wedge f_{MBR2} \wedge f_{MBR3} \wedge f_{MBR7})$ because $MBR4 = MR1$, $MBR5$ implies $MR2$ (i.e. $\forall \sigma.\sigma \models f_{MBR5} \Rightarrow \sigma \models f_{MR2}$), and $MBR6$ implies $MR3$ (i.e. $\forall \sigma.\sigma \models f_{MBR6} \Rightarrow \sigma \models f_{MR3}$). Figure 13 is an example that has a multi-data race bug but no multi-data block race bug because no operation matches p' in the operation block of p . $\square$

$$\phi_{MR} \cap \phi_{MBR} \neq \emptyset$$

Proof

We can prove this statement by using Theorems 1, 2 and 3. $\phi_{DR} \subseteq \phi_{MR}$ (Theorems 2) and $\phi_{BR} \subseteq \phi_{MBR}$ (Theorem 3) imply that $\phi_{DR} \cap \phi_{BR} \subseteq \phi_{MR} \cap \phi_{MBR}$. Thus, $\phi_{MR} \cap \phi_{MBR} \neq \emptyset$ because $\phi_{DR} \cap \phi_{BR} \neq \emptyset$ (Theorem 1). Figure 18 describes the case for $\phi_{MR} \cap \phi_{MBR}$, which has the same code and execution as Figure 10. The multi-data race bug detectors detect two operations p and q as a multi-data race bug because these operations access `balance` and `debt` from different threads without synchronization. In addition, the multi-data block race detection techniques can detect operations p , p' and q as a multi-data block race bug because these operations manipulate associated variables `balance` and `debt` without synchronization. $\square$

$$\phi_{MBR} \not\subseteq \phi_{MR}$$

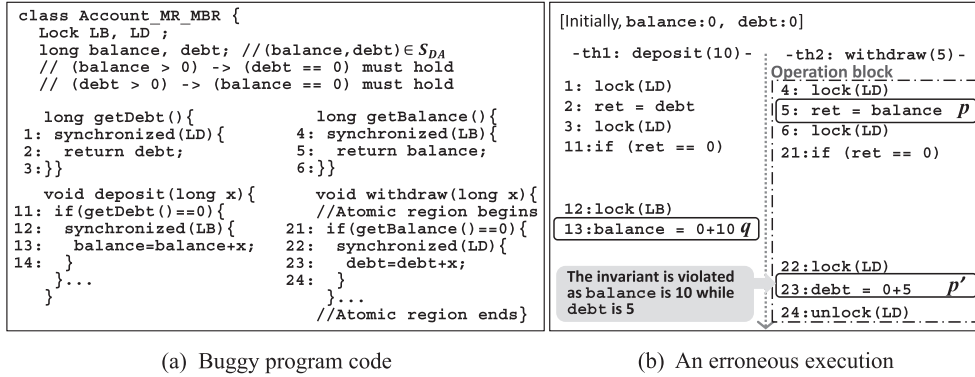


Figure 18. Example of a multi-data race bug detected by multi-data block race bug detectors.

Proof

$\phi_{MBR} \not\subseteq \phi_{MR}$ holds if and only if there exists an execution model that satisfies $F_{MBR} \wedge \neg F_{MR}$. This formula is satisfiable if and only if $F_{MBR} \wedge \neg f_{MR4}$ because $MBR4 = MR1$, $MBR5$ implies $MR2$ (i.e. $\forall \sigma. \sigma \models f_{MBR5} \Rightarrow \sigma \models f_{MR2}$), and $MBR6$ implies $MR3$ (i.e. $\forall \sigma. \sigma \models f_{MBR6} \Rightarrow \sigma \models f_{MR3}$). Figure 11 shows a case where the generated execution model has a multi-data block race bug involving p, q, p' but no multi-data race bug. Because operations p, p', q are ordered with synchronization, multi-data race bug detectors cannot detect these as a race bug. $\square$

9. RELATED WORK

There are a few survey papers on race bugs and race bug detection techniques. Netzer *et al.* [23] formalizes two classes of race conditions, namely, ‘general races’ and ‘data races’. However, the formalism in Netzer *et al.* [23] is too simple to characterize the recently proposed complex race bug detection techniques such as multi-data block race bug detection techniques. Raza [51] surveys a dozen data race detection techniques, all of which belong to the class of data race bug detection techniques (Section 4). Schimmel *et al.* [52] present an empirical evaluation of bug detection capabilities of two data race bug detection tools on real-world concurrent programmes. Wang *et al.* [17] and Qadeer *et al.* [50] present formal specifications of correctness criteria to detect race bugs. Violations of these correctness criteria correspond to the multi-block race bugs (Section 7). Flanagan and Freund present a dynamic analysis framework, RoadRunner [53], that serves as a common platform to facilitate implementation of various race bug detection techniques for Java programmes. They demonstrate that 11 race bug detection techniques could be implemented using RoadRunner.

Compared with those surveys, this paper systematically characterizes and classifies various race bug detection techniques of broader ranges (i.e. 43 race bug detection techniques) by using a formal execution model. We have presented formal definitions of the four classes of race bugs and analysed the 43 race bug detection techniques according to their target race bug definitions. In addition, this survey analyses and demonstrates the relations amongst race bugs of different classes (Section 8). Furthermore, this survey reviews recently developed techniques such as multi-data race bug detection techniques.

There exist several survey papers on concurrent programme analysis topics other than race bug detections for multithreaded programmes. Tchamgoue *et al.* [54] surveys techniques for bug detection and debugging in event-driven programmes and summarizes the terminologies in the techniques. Helmbold *et al.* [55] summarizes the concepts of the race bug detection techniques for parallel programmes and presents a classification with respect to the characteristics of target programme structure (e.g. loop, synchronization operations). Souza *et al.* [56] gives an overview of more than 50 papers on concurrent programme analysis techniques. However, the paper includes only four papers on static/dynamic race bug detection techniques. Fiedor *et al.* [57] describe various concurrency errors including data races, atomicity violations, deadlock and livelocks and the corresponding bug detection techniques. However, the paper mentions only a few techniques briefly

(less than 10 techniques summarized in one page) and does not analyse race bugs and detection techniques in detail. Lu *et al.* [58] report a survey on the concurrency bugs from real software projects such as bug patterns, bug triggering conditions and bug fix strategies. However, the paper does not analyse concurrency bug detection techniques. Bradbury *et al.* [59] compile a set of Java concurrency bug patterns from the literature. Long *et al.* [60] presents a classification of Java concurrency bugs by using a Petri-net diagram. In contrast to the aforementioned works, this survey concentrates on the various race bug detection techniques for multithreaded programmes.

10. CONCLUSION AND FUTURE WORK

We have classified various race bugs and corresponding race bug detection techniques. We first defined a formal execution model as a uniform framework to define each class of race bugs precisely. Then, we classified race bug detection techniques in terms of their utilizing *operation block* and/or *data association* specifications. In addition, we systematically described and compared underlying mechanisms of race bug detection techniques in the same class. We believe that this survey provides researchers with a clear top-down view of various race bug detection techniques and also helps field engineers understand the pitfalls of race bugs through our concrete examples. For future work, we plan to conduct empirical study on various race bug detection techniques by applying them to benchmark programmes and compare their effectiveness and efficiency.

ACKNOWLEDGEMENTS

This work is supported in part by the NRF Mid-career Research Program funded by the MSIP, Korea (NRF-2012R1A2A2A01046172), the IT R&D Program of MKE/KEIT, Korea (10041752), and the ITRC support program funded by MSIP and supervised by the NIPA, Korea (NIPA-2014-H0301-14-1023).

REFERENCES

1. Savage S, Burrows M, Nelson G, Sobalvarro P, Anderson T. Eraser: a dynamic data race detector for multi-threaded programs. *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, Saint Malo, France, October 5–8, 1997; 27–37.
2. Flanagan C, Freund SN, Yi J. Velodrome: a sound and complete dynamic atomicity checker for multithreaded programs. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Tucson, AZ, USA, June 7–13, 2008; 293–303.
3. Lu S, Tucek J, Qing F, Zhou Y. AVIO: detecting atomicity violations via access interleaving invariants. *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, San Jose, California, USA, October 21–25, 2006; 37–48.
4. Sorrentino F, Farzan A, Madhusudan P. PENELOPE: weaving threads to expose atomicity violations. *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, Santa Fe, New Mexico, USA, November 7–11, 2010; 37–46.
5. Zeng R, Sun Z, Liu S, He X. McPatom: a predictive analysis tool for atomicity violation using model checking. *Proceedings of the International Workshop on Model Checking Software (SPIN)*, Oxford, UK, July 23–24, 2012; 191–207.
6. Hammer C, Dolby J, Vaziri M, Tip F. Dynamic detection of atomic-set-serializability violations. *Proceedings of the International Conference on Software Engineering (ICSE)*, Leipzig, Germany, May 10–18, 2008; pp. 231–240.
7. Dias RJ, Pessanha V, Lourenço JM. Precise detection of atomicity violations. *Proceedings of the 8th International Conference on Hardware and Software: Verification and Testing (HVC)*, Haifa, Israel, November 6–8, 2012; pp. 8–23.
8. Yi J, Sadowski C, Flanagan C. Cooperative reasoning for preemptive execution. *Proceedings of the ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*, San Antonio, Texas, USA, February 12–16, 2011; 147–156.
9. Lucia B, Devietti J, Strauss K, Ceze L. Atom-Aid: detecting and surviving atomicity violations. *Proceedings of the Annual International Symposium on Computer Architecture (ISCA)*, Washington, DC, USA, June 21–25, 2008; 277–288.
10. Letko Z, Vojnar T, Křena B. AtomRace: data race and atomicity violation detector and healer. *Proceedings of the Workshop on Parallel and Distributed Systems: Testing, Analysis, and Debugging (PADTAD)*, Seattle, Washington, USA, July 20–21, 2008; 7:1–7:10.
11. Chew L, Lie D. Kivati: fast detection and prevention of atomicity violations. *Proceedings of the European Conference on Computer Systems (EUROSYS)*, Paris, France, April 13–16, 2010; 307–320.

12. Sumner WN, Hammer C, Dolby J. Marathon: detecting atomic-set serializability violations with conflict graphs. *Proceedings of the International Conference on Runtime Verification (RV)*, San Francisco, CA, USA, September 27–30, 2011; 161–176.
13. Lucia B, Ceze L, Strauss K. ColorSafe: architectural support for debugging and dynamically avoiding multi-variable atomicity violations. *Proceedings of the Annual International Symposium on Computer Architecture (ISCA)*, Saint-Malo, France, June 19–23, 2010; 222–233.
14. Wang L, Stoller SD. Runtime analysis for atomicity for multithreaded programs. *IEEE Transactions on Software Engineering* 2006; **32**(2):99–110.
15. Chen Q, Wang L, Yang Z, Stoller SD. HAVE: integrated dynamic and static analysis for atomicity violations. *Proceedings of the International Conference on Fundamental Approaches to Software Engineering (FASE)*, York, UK, March 22–29, 2009; 425–439.
16. Artho C, Havelund K, Biere A. Using block-local atomicity to detect stale-value concurrency errors. *Proceedings of the International Symposium on Automated Technology for Verification And Analysis (ATVA)*, Taipei, Taiwan, October 31 - November 3, 2004; 150–164.
17. Wang L, Stoller SD. Accurate and efficient runtime detection of atomicity errors in concurrent programs. *Proceedings of the ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*, New York, New York, USA, March 29–31, 2006; 137–146.
18. Artho C, Havelund K, Biere A. High-level data races. *Proceedings of the International Workshop on Verification and Validation of Enterprise Information Systems (VVIIES)*, Angers, France, April 22, 2003; 82–93.
19. von Praun C, Gross T. Static detection of atomicity violations in object-oriented programs. *Journal of Object Technology* 2004; **3**(2):103–122.
20. Pozniansky E, Schuster A. MultiRace: efficient on-the-fly data race detection in multithreaded C++ programs. *Concurrency and Computation: Practice and Experience* 2007; **19**(3):327–340.
21. von Praun C, Gross TR. Object race detection. *Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, Tampa Bay, Florida, USA, October 14–18, 2001; 70–82.
22. Lu S, Park S, Hu C, Ma X, Jiang W, Li Z, Popa RA, Zhou Y. MUVI: automatically inferring multi-variable access correlations and detecting related semantic and concurrency bugs. *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, Stevenson, Washington, USA, October 14–17, 2007; 103–116.
23. Netzer RHB, Miller BP. What are race conditions?: some issues and formalizations. *ACM Letters on Programming Languages and Systems* 1992; **1**(1):74–88.
24. Park S, Lu S, Zhou Y. CTrigger: exposing atomicity violation bugs from their hiding places. *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Washington, DC, USA, March 7–11, 2009; 25–36.
25. Vaziri M, Tip F, Dolby J. Associating synchronization constraints with data in an object-oriented language. *Proceedings of the Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, Charleston, South Carolina, USA, January 11–13, 2006; 334–345.
26. Flanagan C, Freund SN. FastTrack: efficient and precise dynamic race detection. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Dublin, Ireland, June 15–20, 2009; 121–133.
27. Christiaens M, Bosschere KD. TRaDe, a topological approach to on-the-fly race detection in Java programs. *Proceedings of the Symposium on JavaTM Virtual Machine Research and Technology Symposium (JVM)*, Monterey, California, USA, April 23–24, 2001; 15:1–15:12.
28. Bodden E, Havelund K. Aspect-oriented race detection in Java. *IEEE Transactions on Software Engineering* 2010; **36**(4):509–527.
29. Engler D, Ashcraft K. RacerX: effective, static detection of race conditions and deadlocks. *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, Bolton Landing, New York, USA, October 19–22, 2003; 237–252.
30. Xie X, Xue J, Zhang J. ACCULOCK: accurate and efficient detection of data races. *Software-Practice and Experience* 2012; **43**(5):543–576.
31. Choi JD, Lee K, Loginov A, O’Callahan R, Sarkar V, Sridharan M. Efficient and precise datarace detection for multithreaded object-oriented programs. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Berlin, Germany, June 17–19, 2002; 258–269.
32. Mekkat V, Holey A, Zhai A. Accelerating data race detection utilizing on-chip data-parallel cores. *Proceedings of the International Conference on Runtime Verification (RV)*, Rennes, France, September 24–27, 2013; 201–218.
33. Muzahid A, Otsuki N, Torrellas J. AtomTracker: a comprehensive approach to atomic region inference and violation detection. *Proceedings of the annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Atlanta, Georgia, USA, Dec 4–8, 2010; 287–297.
34. Marino D, Musuvathi M, Narayanasamy S. LiteRace: effective sampling for lightweight data-race detection. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Dublin, Ireland, 2009.
35. O’Callahan R, Choi JD. Hybrid dynamic data race detection. *Proceedings of the ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*, San Diego, California, USA, June 11–13, 2003; 167–178.

36. Yu Y, Rodeheffer T, Chen W. RaceTrack: efficient detection of data race conditions via adaptive tracking. *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, Brighton, UK, October 23–26, 2005; 221–234.
37. Sheng T, Vachharajani N, Eranian S, Hundt R, Chen W, Zheng W. RACEZ: a lightweight and non-invasive race detection tool for production applications. *Proceedings of the International Conference on Software Engineering (ICSE)*, Honolulu, Hawaii, USA, May 21–28, 2011; 401–410.
38. Li D, Srisa-an W, Dwyer MB. SOS: saving time in dynamic race detection with stationary analysis. *Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, Portland, Oregon, USA, October 22–27, 2011; 35–50.
39. Serebryany K, Iskhodzhanov T. ThreadSanitizer – data race detection in practice. *Proceedings of the Workshop on Binary Instrumentation and Applications (WBIA)*, New York, New York, USA, December 12, 2009; 62–71.
40. Shi Y, Park S, Yin Z, Lu S, Zhou Y, Chen W, Zheng W. Do I use the wrong definition? DeFuse: definition-use invariants for detecting concurrency and sequential bugs. *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA)*, Reno/Tahoe, Nevada, USA, October 17–21, 2010; 160–174.
41. Park S, Vuduc RW, Harrold MJ. Falcon: fault localization in concurrent programs. *Proceedings of the International Conference on Software Engineering (ICSE)*, Cape Town, May 1–8, 2010; 245–254.
42. Naik M, Aiken A, Whaley J. Effective static race detection for Java. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Ottawa, Ontario, Canada, June 11–14, 2006; 308–319.
43. Flanagan C, Freund SN. Type-based race detection for java. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Vancouver, BC, Canada, June 18–21, 2000; 219–232.
44. Young JW, Jhala R, Lerner S. RELAY: static race detection on millions of lines of code. *Proceedings of the Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC-FSE)*, Dubrovnik, Croatia, September 3–7, 2007; 205–214.
45. Chen F, Rosu G. Parametric and sliced causality. *Proceedings of the International Conference on Computer Aided Verification (CAV)*, Berlin, Germany, July 3–7, 2007; 240–253.
46. Burnim J, Sen K, Stergiou C. Testing concurrent programs on relaxed memory models. *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, Toronto, Canada, July 17–21, 2011; 122–132.
47. Adve S. Data races are evil with no exceptions: Technical perspective. *Communications of the ACM* 2010; **53**(11):84–84.
48. Boehm HJ. Position paper: nondeterminism is unavoidable, but data races are pure evil. *Proceedings of ACM Workshop on Relaxing Synchronization for Multicore and Manycore Scalability (RACES)*, Tucson, Arizona, USA, October 21, 2012; 9–14.
49. Flanagan C, Freund SN. Atomizer: a dynamic atomicity checker for multithreaded programs. *Proceedings of the Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, Venice, Italy, January 14–16, 2004; 256–267.
50. Qadeer S, Tasiran S. Runtime verification of concurrency-specific correctness. *Software Tools for Technology Transfer* 2012; **14**(3):291–305.
51. Raza A. A review of race detection mechanisms. *Proceedings of the International Computer Science Conference on Theory and Applications (CSR)*, St. Petersburg, Russia, June 8–12, 2006; 534–543.
52. Schimmel J, Molitorisz K, Tichy WF. An evaluation of data race detectors using bug repositories. *Proceedings of the Federated Conference on Computer Science and Information Systems*, Krakow, Poland, September 8–11, 2013; 1361–1364.
53. Flanagan C, Freund SN. The RoadRunner dynamic analysis framework for concurrent programs. *Proceedings of Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, Toronto, Canada, June 5–6, 2010; 1–8.
54. Tchamgoue GM, Ha OK, Kim KH, Jun YK. A taxonomy of concurrency bugs in event-driven programs. *Proceedings of International Conference on Advanced Software Engineering and Its Applications (ASEA)*, Jeju, Korea, December 8–10, 2011; 437–450.
55. Helmbold DP, McDowell CE. A taxonomy of race conditions. *Journal of Parallel and Distributed Computing* 1996; **33**(2):159–164.
56. Souza SRS, Brito MAS, Silva RA, Souza PSL, Zaluska E. Research in concurrent software testing: a systematic review. *Proceedings of the Workshop on Parallel and Distributed Systems: Testing, Analysis, and Debugging (PADTAD)*, Toronto, Canada, July 17, 2011; 1–5.
57. Fiedor J, Křena B, Letko Z, Vojnar T. A uniform classification of common concurrency errors. *Proceedings of the International Conference on Computer Aided Systems Theory (EUROCAST)*, Las Palmas de Gran Canaria, Spain, February 6–11, 2012; 519–526.
58. Lu S, Park S, Seo E, Zhou Y. Learning from mistakes: a comprehensive study on real world concurrency bug characteristics. *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Seattle, Washington, USA, March 1–5, 2008; 329–339.
59. Bradbury JS, Jalbert K. Defining a catalog of programming and anti-patterns for concurrent java. *Proceedings of the International Workshop on Software Patterns and Quality (SPAQu)*, Orlando, Florida, USA, October 25, 2009; 6–11.
60. Long B, Strooper P. A classification of concurrency failures in Java components. *Proceedings of the Workshop on Parallel and Distributed Systems: Testing, Analysis, and Debugging (PADTAD)*, Nice, France, April 22–26, 2003; 3:1–3:8.